SSA 形式に対する最適化

SSA 形式：データフロー解析ずみのプログラム

構文的にフローが表現されている ⇒ 最適化を効果的に適用できる：更新のチェックが不要
例：DAG の構成が容易

- 不用コードの削除
- 定数伝播
- コピー伝播
- 共通部分式の削除
- ループ不変コードの移動

SSA 最適化例 [佐々:06]

```
a=1;                          a1=1;
b=2;                          b1=2;
x=3;                          x1=3;
y=a+b;                        y1=a1+b1;
c=x+y;                        c1=x1+y1;
if(a<10)                      if(a1<10)
   z=a+b;                        z1=a1+b1;
else                          else
   z=a+b;                        z2=a1+b1;
d=x+z;                        z3=φ(z1,z2);
println(d);          ⇒        d1=x1+z3;
                              println(d1);
```

SSA 形式へ変換

SSA 最適化例 [佐々:06]

```
a1=1;
b1=2;
x1=3;
y1=a1+b1;
c1=x1+y1;
if(a1<10)
   z1=a1+b1;
else
   z2=a1+b1;
z3=φ(z1,z2);
d1=x1+z3;
println(d1);
```

⇒

```
a1=1;
b1=2;
x1=3;
y1=a1+b1;
c1=x1+y1;
if(a1<10)
   z1=y1;
else
   z2=y1;
z3=y1(=φ(y1,y1));
d1=x1+z3;
println(d1);
```

共通部分式除去

SSA 最適化例 [佐々:06]

```
a1=1;
b1=2;
x1=3;
y1=a1+b1;
c1=x1+y1;
if(a1<10)
   z1=y1;
else
   z2=y1;
z3=y1(=φ(y1,y1));
d1=x1+z3;
println(d1);
```

⇒

```
a1=1;
b1=2;
x1=3;
y1=a1+b1;
c1=x1+y1;
if(a1<10)
   z1=y1;
else
   z2=y1;
z3=y1;
d1=x1+y1;
println(d1);
```

コピー伝播

SSA 最適化例 [佐々:06]

```
a1=1;                          a1=1;
b1=2;                          b1=2;
x1=3;                          x1=3;
y1=a1+b1;                      y1=3;
c1=x1+y1;                      c1=6;
if(a1<10)                      if(a1<10)
   z1=y1;                         z1=y1;
else                           else
   z2=y1;                         z2=y1;
z3=y1;                         z3=y1;
d1=x1+y1;                      d1=c1;
println(d1);          ⇒        println(d1);
```

共通部分式除去、定数伝播

SSA 最適化例 [佐々:06]

```
a1=1;
b1=2;
x1=3;
y1=3;
c1=6;
if(a1<10)
   z1=y1;
else
   z2=y1;
z3=y1;
d1=c1;
println(d1);
```

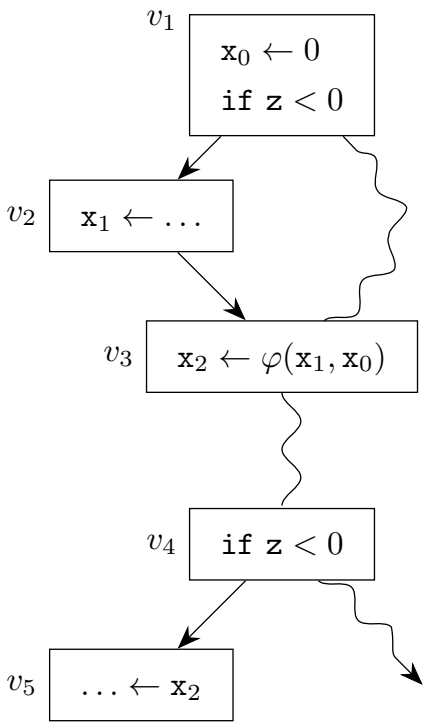
$\Rightarrow$

```
a1=1;
b1=2;
x1=3;
y1=3;
c1=6;
z3=y1;
println(6);
```

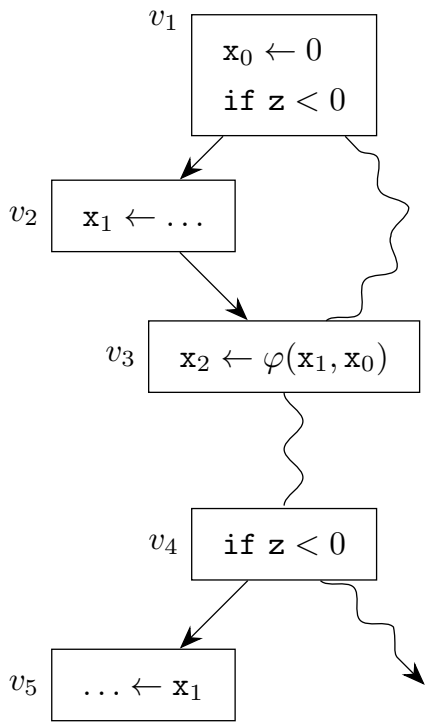
コピー伝播、定数伝播、恒等式

最適化の条件

最適化を行うときに支配関係を壊さないようにする。



(a) 最適化前



(b) 最適化後

$z < 0$ の評価は同じ方向にしかブランチしない

$\Rightarrow$ $z < 0$ ならば 3 の ϕ 関数はいつも x_1 を x_2 へ代入

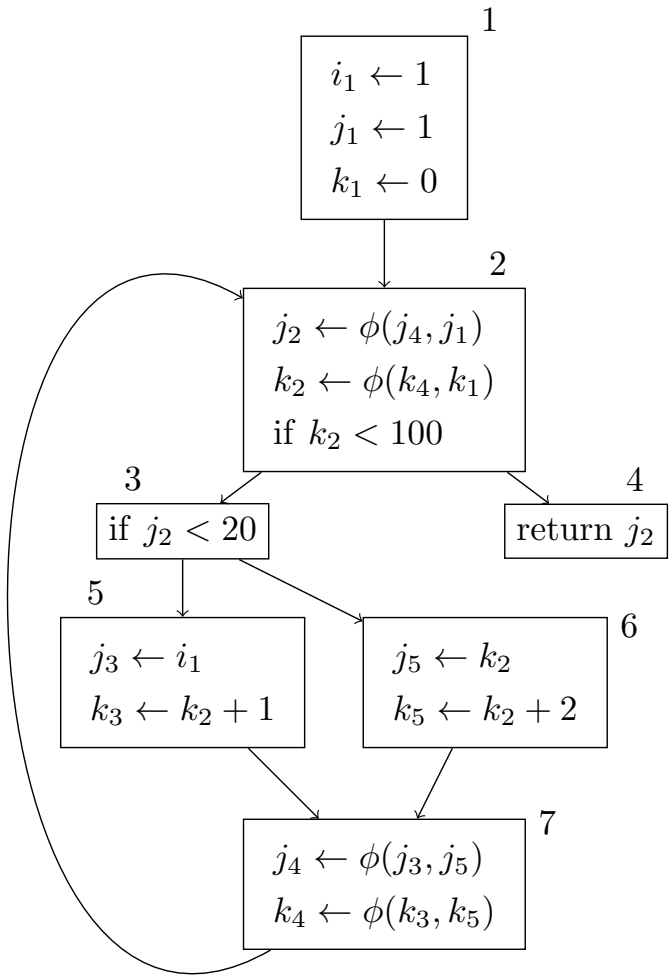
5 において $z < 0$ で、x_1 を参照に用いると支配関係が壊れる

厳密 SSA 形式：**定義が一意的＋支配関係**

条件付き定数伝播 [Tigerbook]

```
i = 1
j = 1
k = 0
while (k<100) do
  if (j<20) then
    j = i
    k = k + 1
  else
    j = k
    k = k + 2
  fi
od
return j
```

⇒



6 のブロックは実行されるか？

アルゴリズム

$\mathcal{B}[n] = \mathit{true}$ ならばブロック n は実行される可能性あり
$\mathcal{V}[x]$ 定数 c, 決まらない $\top$, 初期化されない $\bot$

$\phi(x_1, \ldots, x_n)$ において，x_i のフロー元となる直近のブロックを b_i と書く．

初期化：$\mathcal{B}[n] = \mathit{false}$ $(1 \leq n \leq N)$, $\mathcal{V}[x] = \bot$ $(x \in X)$

Step 1. 実行の引数となる変数 $x \in P$ について，$\mathcal{V}(x) \leftarrow \top$

Step 2. $\mathcal{B}[1] \leftarrow \mathit{true}$

$\mathcal{B}$ と $\mathcal{V}$ が変化しなくなるまで Step 3 $\sim$ Step 10 を繰り返す．

Step 3. $\mathcal{B}[n] = \mathit{true}$ の次のブロックが唯一の m のとき，$\mathcal{B}[m] \leftarrow \mathit{true}$

Step 4. $z \leftarrow x \mathit{\ op\ } y$ に対して $\mathcal{V}[x] = c_1$ かつ $\mathcal{V}[y] = c_2$ のとき，$\mathcal{V}[z] \leftarrow c_1 \mathit{\ op\ } c_2$

Step 5. $z \leftarrow x \mathit{\ op\ } y$ に対して $\mathcal{V}[x] = \top$ または $\mathcal{V}[y] = \top$ のとき，$\mathcal{V}[z] \leftarrow \top$

Step 6. $y \leftarrow \phi(x_1, \ldots, x_n)$ に対して，$\mathcal{V}[x_i] = c_i$, $\mathcal{V}[x_j] = c_j$, $c_i \neq c_j$ となる i, j が存在し，$\mathcal{B}[b_i] = \mathcal{B}[b_j] = \mathit{true}$ のとき，$y \leftarrow \top$

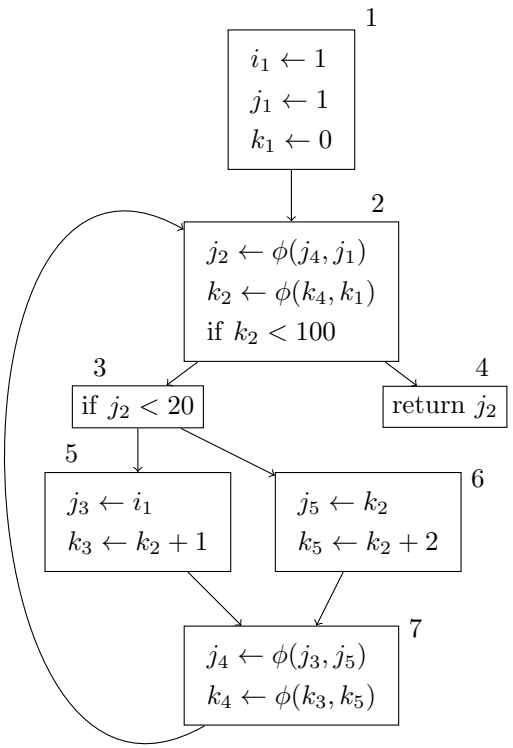
Step 7. $y \leftarrow \phi(x_1, \ldots, x_n)$ に対して，$\mathcal{V}[x_i] = \top$ となる i が存在するならば $\mathcal{V}[y] = \top$

Step 8. $y \leftarrow \phi(x_1, \ldots, x_n)$ に対して，$\mathcal{B}[b_i] = \mathit{true}$ である x_i ですべて $\mathcal{V}[x_i] = c$ であり，$\mathcal{B}[b_j] = \mathit{false}$ である x_j ですべて $\mathcal{V}[x_j] = \bot$ ならば，$\mathcal{V}[y] \leftarrow c$

Step 9. **if** $x < y$ **goto** B_i **else** B_j に対して $\mathcal{V}[x] = \top$ または $\mathcal{V}[y] = \top$ ならば $\mathcal{B}[i] \leftarrow \mathit{true}$, $\mathcal{B}[j] \leftarrow \mathit{true}$

Step 10. **if** $x < y$ **goto** B_i **else** B_j に対して $\mathcal{V}[x] = c_1$, $\mathcal{V}[y] = c_2$ のとき，$c_1 < c_2$ ならば $\mathcal{B}[i] \leftarrow \mathit{true}$, $c_1 \not< c_2$ ならば $\mathcal{B}[j] \leftarrow \mathit{true}$.

条件付き定数伝播

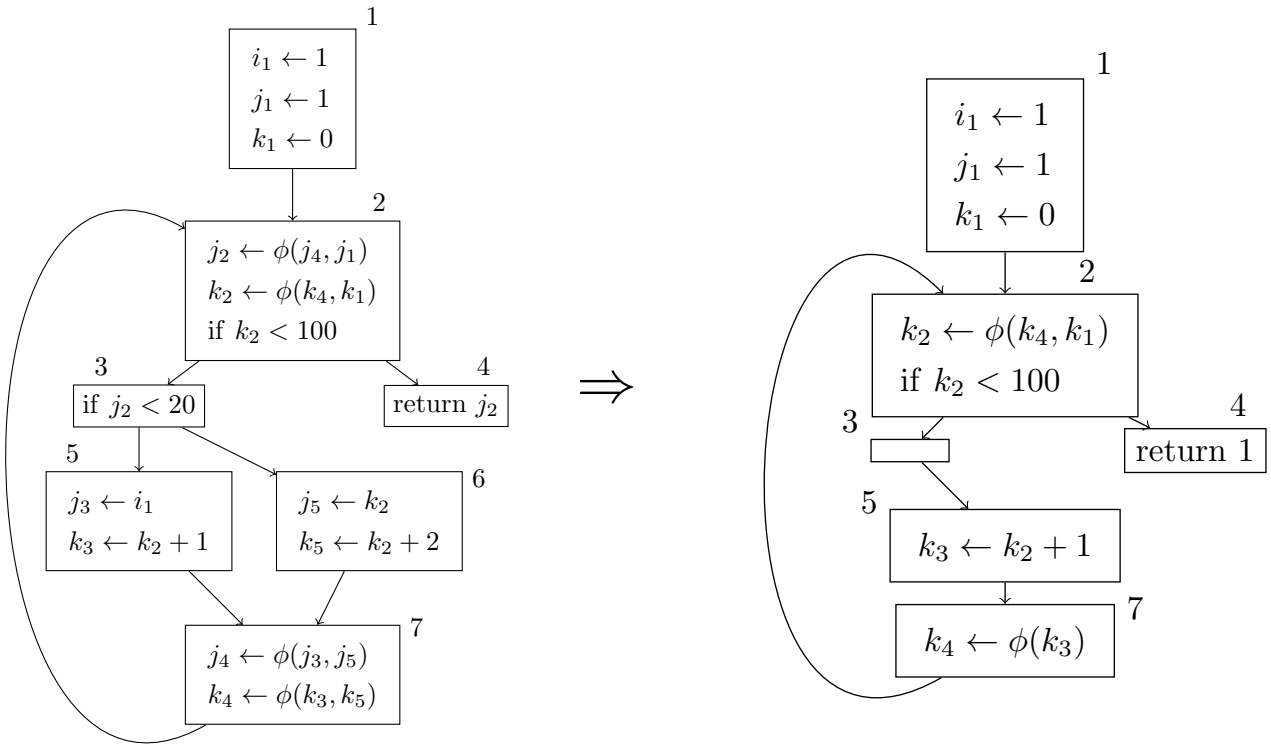


B	$\mathcal{E}[B]$
1	*true*
2	*true*
3	*true*
4	*true*
5	*true*
6	*false*
7	*true*

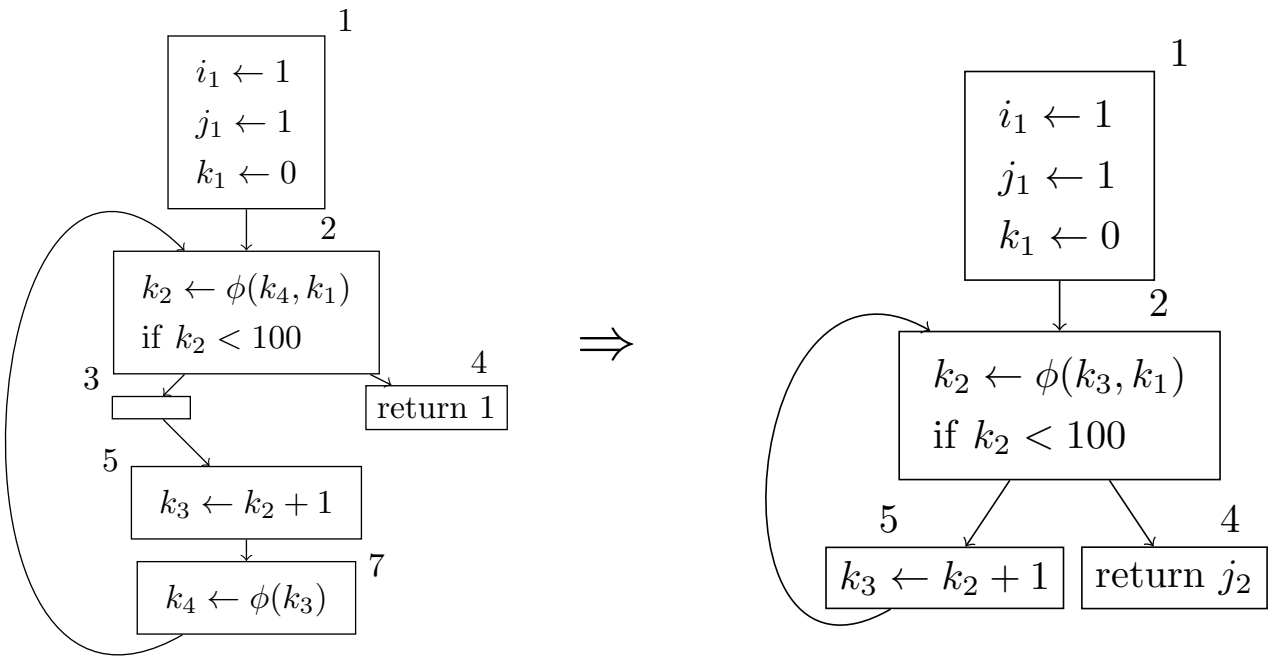
x	$\mathcal{V}[x]$
i_1	1
j_1	1
j_2	1
j_3	1
j_4	1
j_5	$\bot$
k_1	0
k_2	$\top$
k_3	$\top$
k_4	$\top$
k_5	$\bot$

j は定数が伝播して，$j < 20$ の判定がいつも成立．k は $\top$ なので $k < 100$ の判定は実行時

条件付き定数伝播

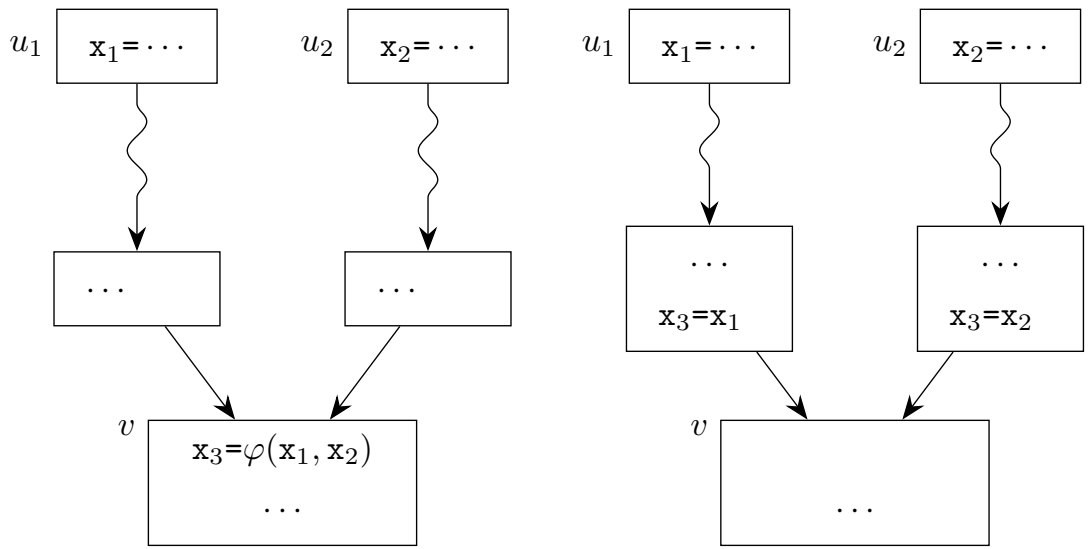

1
$i_1 \leftarrow 1$
$j_1 \leftarrow 1$
$k_1 \leftarrow 0$
2
$j_2 \leftarrow \phi(j_4, j_1)$
$k_2 \leftarrow \phi(k_4, k_1)$
if $k_2 < 100$
3
if $j_2 < 20$
4
return j_2
5
$j_3 \leftarrow i_1$
$k_3 \leftarrow k_2 + 1$
6
$j_5 \leftarrow k_2$
$k_5 \leftarrow k_2 + 2$
7
$j_4 \leftarrow \phi(j_3, j_5)$
$k_4 \leftarrow \phi(k_3, k_5)$
⇒
1
$i_1 \leftarrow 1$
$j_1 \leftarrow 1$
$k_1 \leftarrow 0$
2
$k_2 \leftarrow \phi(k_4, k_1)$
if $k_2 < 100$
3
4
return 1
5
$k_3 \leftarrow k_2 + 1$
7
$k_4 \leftarrow \phi(k_3)$

条件付き定数伝播


1
$i_1 \leftarrow 1$
$j_1 \leftarrow 1$
$k_1 \leftarrow 0$
2
$k_2 \leftarrow \phi(k_4, k_1)$
if $k_2 < 100$
3
4
return 1
5
$k_3 \leftarrow k_2 + 1$
7
$k_4 \leftarrow \phi(k_3)$
⇒
1
$i_1 \leftarrow 1$
$j_1 \leftarrow 1$
$k_1 \leftarrow 0$
2
$k_2 \leftarrow \phi(k_3, k_1)$
if $k_2 < 100$
5
4
$k_3 \leftarrow k_2 + 1$
return j_2

逆変換アルゴリズム

先行ブロックにフローごとの先行ブロックの最後が goto 文の場合は、goto 文の直前に代入文を挿入する。



逆変換アルゴリズム

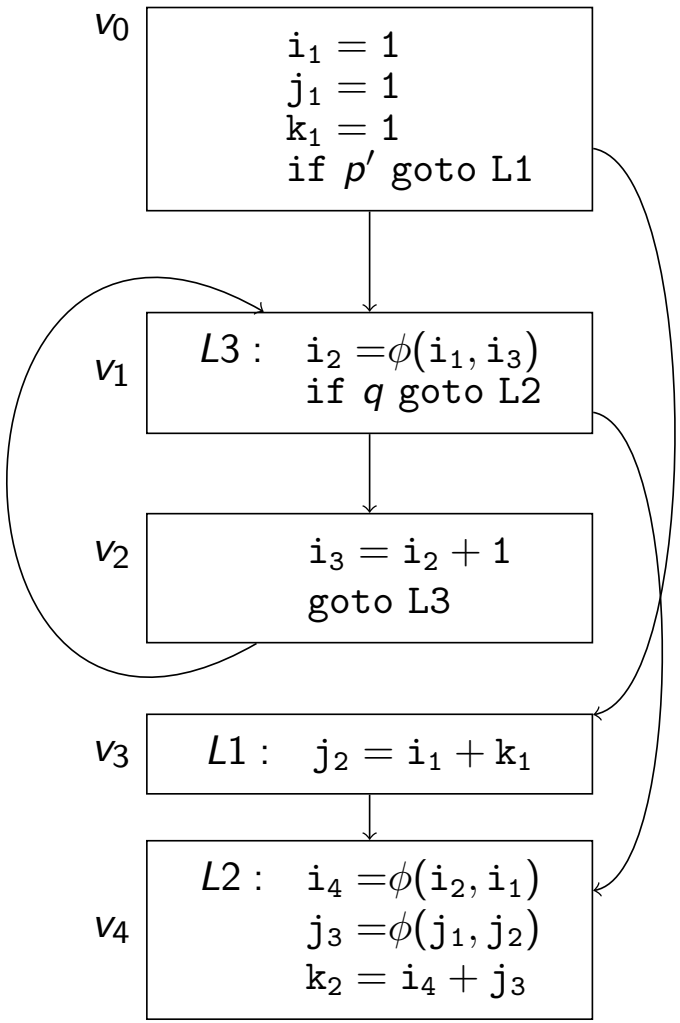
先行ブロックにフローごとの先行ブロックの最後が goto 文の場合は、goto 文の直前に代入文を挿入する。

1. フローグラフの変換
   ϕ 関数を持つブロックに対して挿入条件を満たさないまで、代入文を挿入する。

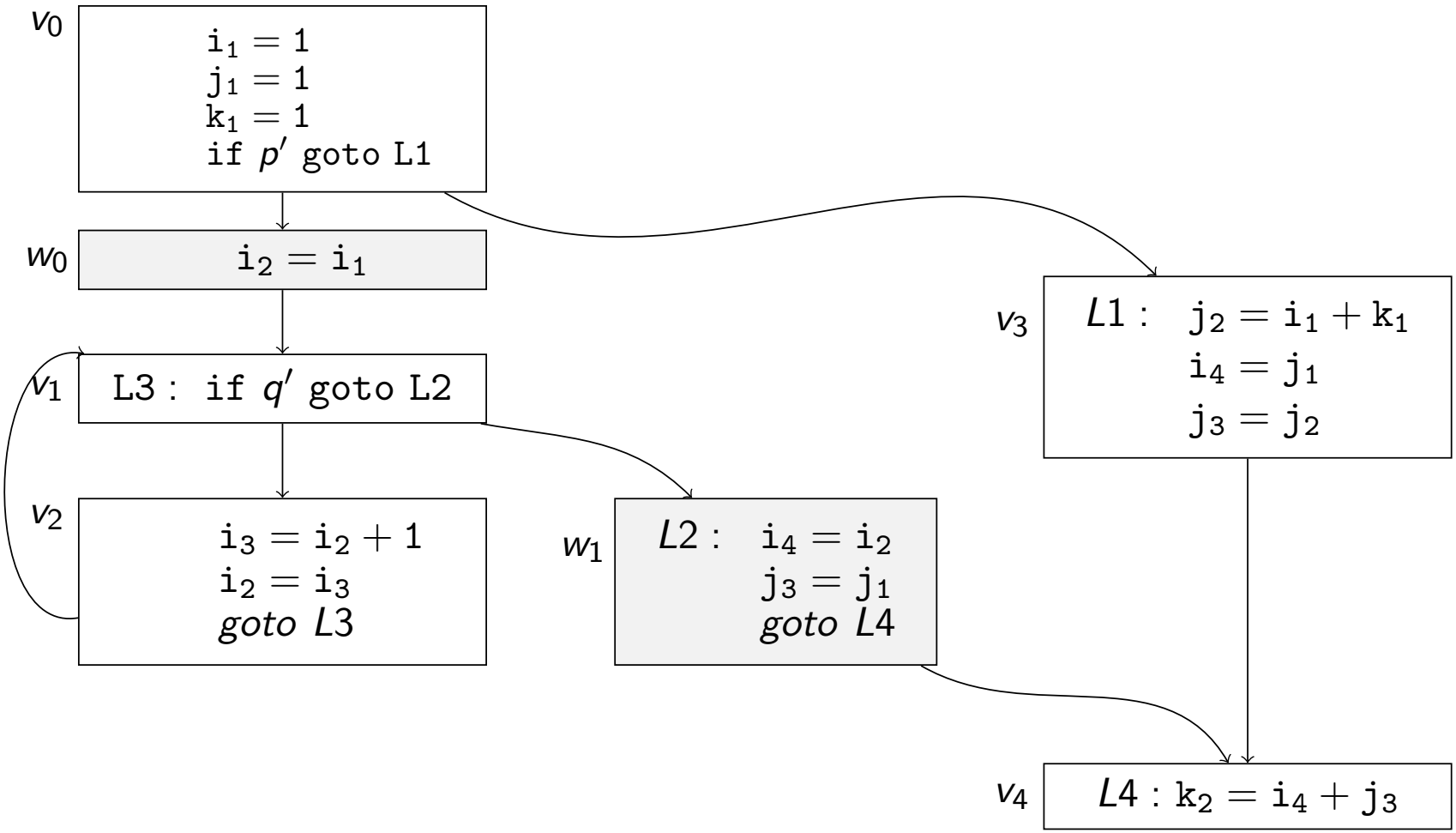
   挿入条件：ϕ 関数を持つブロックが複数の先行頂点を持ち、そのうちの一つが複数の後続頂点を持つ。

2. ϕ 関数の消去

逆変換例


v_0
$i_1 = 1$
$j_1 = 1$
$k_1 = 1$
if p' goto L1
v_1
$L3:$ $i_2 = \phi(i_1, i_3)$
if q goto L2
v_2
$i_3 = i_2 + 1$
goto L3
v_3
$L1:$ $j_2 = i_1 + k_1$
v_4
$L2:$ $i_4 = \phi(i_2, i_1)$
$j_3 = \phi(j_1, j_2)$
$k_2 = i_4 + j_3$

逆変換例


v_0
$i_1 = 1$
$j_1 = 1$
$k_1 = 1$
if p' goto L1
w_0
$i_2 = i_1$
v_1
L3 : if q' goto L2
v_2
$i_3 = i_2 + 1$
$i_2 = i_3$
goto $L3$
w_1
$L2$: $i_4 = i_2$
$j_3 = j_1$
goto $L4$
v_3
$L1$: $j_2 = i_1 + k_1$
$i_4 = j_1$
$j_3 = j_2$
v_4
$L4 : k_2 = i_4 + j_3$

φ 関数のない SSA 形式

```
     i1 = 1
     j1 = 1
     k1 = 1
     if p' goto L1
     i2 = i1
L3:  if q' goto L2
     i3 = i2 + 1
     i2 = i3
     goto L3
L2:  i4 = i2
     j3 = j1
     goto L4
L1:  j2 = i1 + k1
     i4 = j1
     j3 = j2
L4:  k2 = i4 + j3
```